

Демонстрація до лабораторної роботи № 5

Виявлення об'єктів (Object Detection) за допомогою YOLO

Цей ноутбук — приклад для показу на парі. Ми пройдемося по всіх пунктах завдання лабораторної: inference попередньо навченої моделі **YOLO** (You Only Look Once), обробка зображень і відео, побудова bounding boxes, **NMS** (Non-Maximum Suppression — придушення немаксимумів), обчислення **IoU** (Intersection over Union — перетин над об'єднанням) та **mAP** (mean Average Precision — середня точність, усереднена по класах), а також дослідницька частина — вплив confidence threshold (порогу впевненості) на Precision/Recall та залежність FPS (Frames Per Second — кадрів за секунду) від роздільної здатності.

Модель: ми використовуємо `YOLOv8` (родина моделей YOLO версії 8) з бібліотеки `ultralytics`, попередньо навчену на датасеті **COCO** (Common Objects in Context — 80 класів побутових об'єктів).

1. Налаштування середовища

Завантажуємо попередньо навчену модель `yolov8n.pt` ("n" = nano, найменша й найшвидша версія). Приклад-зображення (`bus.jpg`) йде разом із бібліотекою `ultralytics`, тож не потрібно нічого завантажувати додатково.

Модель завантажено: `yolov8n.pt`
Кількість класів (COCO): 80
Приклади класів: ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light'] ...

2. Inference на статичному зображенні

Запускаємо модель на зображенні з параметром `conf` (confidence threshold — мінімальний рівень впевненості, при якому детекція вважається "справжньою"). Модель повертає для кожного знайденого об'єкта: bounding box (координати прямокутника), клас та рівень впевненості.

Час inference: 580.4 мс
Знайдено об'єктів: 6

bus	conf=0.87	box=[22.9, 231.3, 805.0, 756.8]
person	conf=0.87	box=[48.6, 398.6, 245.3, 902.7]
person	conf=0.85	box=[669.5, 392.2, 809.7, 877.0]
person	conf=0.83	box=[221.5, 405.8, 345.0, 857.5]
person	conf=0.26	box=[0.0, 550.5, 63.0, 873.4]
stop sign	conf=0.26	box=[0.1, 254.5, 32.6, 324.9]

Візуалізуємо результат — модель сама вміє малювати bounding boxes з підписами класу та рівня впевненості поверх зображення (метод `.plot()`).

YOLOv8n: результат детекції (conf ≥ 0.25)



3. IoU (Intersection over Union — перетин над об'єднанням)

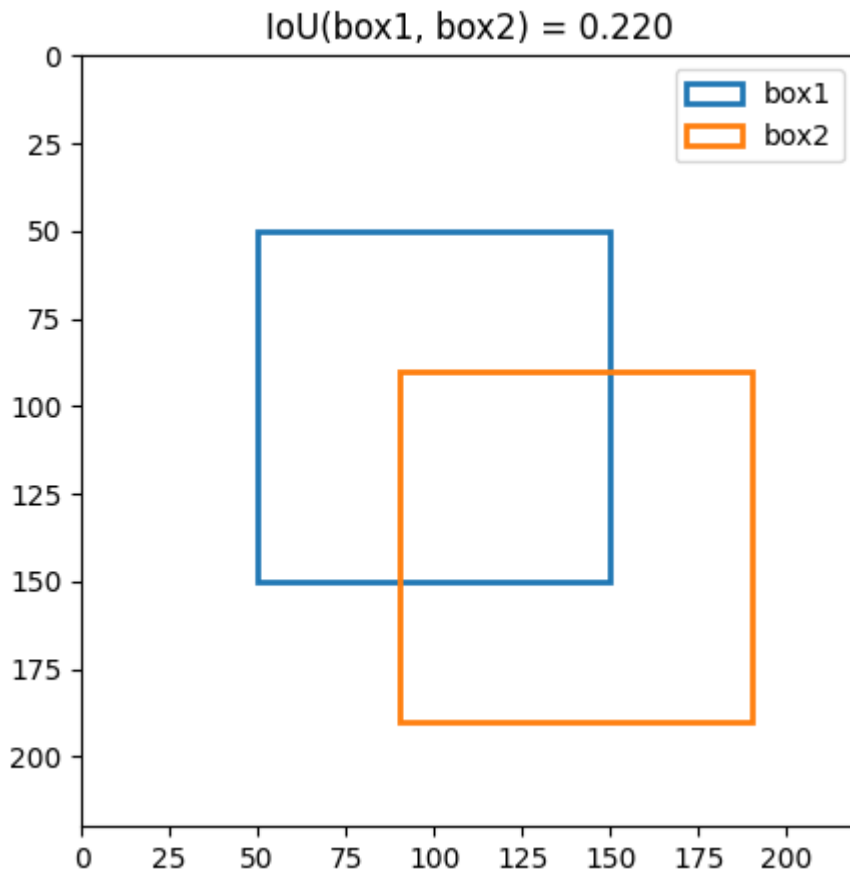
IoU — базова метрика, яка показує, наскільки два прямокутники (наприклад, передбачений і еталонний bounding box) збігаються. Формула:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

IoU = 1 означає ідеальний збіг, IoU = 0 — прямокутники не перетинаються взагалі. У завданнях детекції зазвичай вважають детекцію "правильною" (True Positive), якщо IoU з еталонним боксом перевищує певний порог (типово 0.5).

`IoU(box1, box2) = 0.22`

`IoU(box1, box3) = 0.0`



4. NMS (Non-Maximum Suppression — придушення немаксимумів)

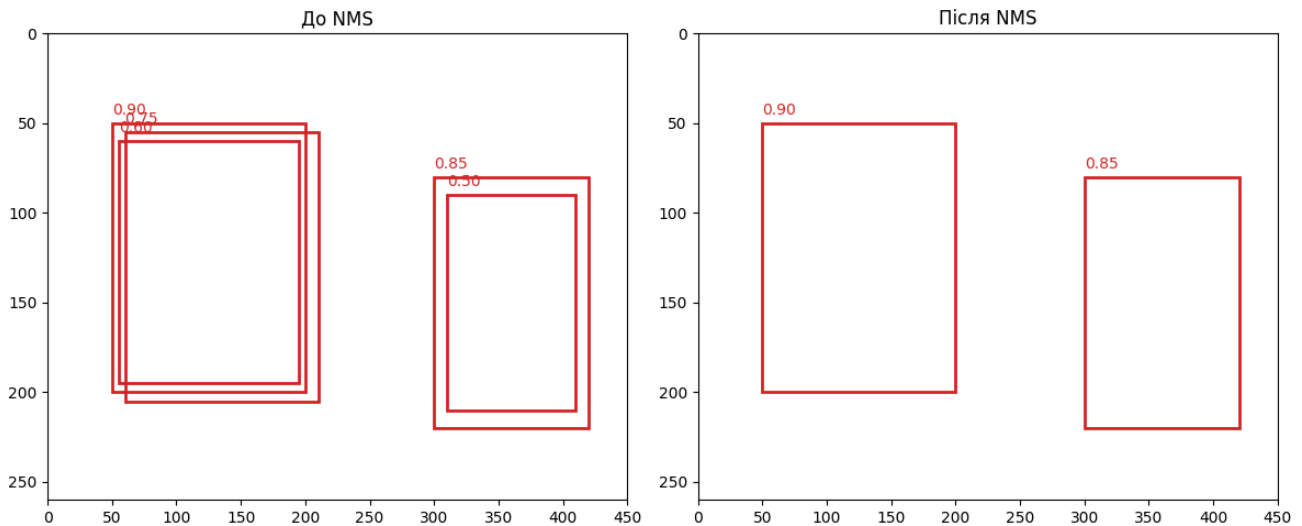
Модель детекції зазвичай генерує декілька перекривних bounding boxes навколо одного й того самого об'єкта (з різною впевненістю). NMS — алгоритм, який залишає лише найкращий бокс серед групи перекривних, видаляючи "дублікати":

1. Відсортувати всі бокси за впевненістю (confidence) — від найбільшої до найменшої.
2. Взяти бокс з найвищою впевненістю, додати його в результат.
3. Видалити всі інші бокси, чий IoU з обраним боксом перевищує заданий порог (`iou_threshold`).
4. Повторювати, поки не залишиться жодного боксу.

Продемонструємо на штучному прикладі — 5 боксів, з яких два "кластери" по 2-3 дублікати навколо одного об'єкта.

До NMS: боксів = 5

Після NMS: боксів = 2 -> залишені індекси: [0, 3]



На реальному зображенні NMS вбудований у `model.predict()` через параметр `iou` (порог IoU для придушення дублікатів). Подивимось, як кількість фінальних боксів залежить від цього порогу — при дуже високому порозі (0.9) майже дублікати починають "виживати".

```

iou_threshold = 0.3: залишилось боксів = 6
iou_threshold = 0.5: залишилось боксів = 6
iou_threshold = 0.7: залишилось боксів = 6
iou_threshold = 0.9: залишилось боксів = 7

```

5. Обробка відео

Обробляємо відеопотік кадр за кадром: читаємо кадр -> запускаємо детекцію -> малюємо бокси -> записуємо у вихідний файл. Для демонстрації згенеруємо коротке синтетичне відео (панорама по зображенню `bus.jpg`), оскільки веб-камера тут не використовується.

Синтетичне відео створено: `synthetic.mp4`

Опрацьовано 20 кадрів за 0.47 с -> 42.9 FPS

Всього детекцій за все відео: 52

Приклади кадрів відео з детекцією об'єктів



6. mAP (mean Average Precision) та Precision/Recall на розміченому наборі даних

Щоб чесно обчислити Precision, Recall і mAP, потрібні зображення з **еталонною розміткою** (ground truth). Для демонстрації використаємо **coco128** — маленький (128 зображень) підмножина датасету COCO з готовою розміткою, яку **ultralytics** вміє завантажувати автоматично. Бібліотека сама рахує:

- **Precision** (точність) — частка правильних детекцій серед усіх зроблених;
- **Recall** (повнота) — частка знайдених об'єктів серед усіх, що насправді є на зображенні;
- **mAP50** — mean Average Precision при порозі IoU = 0.5;
- **mAP50-95** — усереднення mAP по порогах IoU від 0.5 до 0.95.

```
Ultralytics 8.4.147 🚀 Python-3.9.6 torch-2.8.0 CPU (Apple M3 Max)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 131.1±52.0 MB/s, size: 47.3 KB)
val: Scanning /Users/vmiachkov/datasets/coco128/labels/train2017.cache... 126 images, 2 backgrounds, 0 corrupt: 100% ██████████ 128/128 25.6Mit/s 0.0s
Class      Images  Instances  Box(P          R              mAP50  mAP
50-95): 100% ██████████ 8/8 1.4s/it 10.8s.6ss
          all        128        929        0.677        0.499        0.466
0.363
Speed: 0.2ms preprocess, 82.6ms inference, 0.0ms loss, 0.1ms postprocess per image
Precision (mean, усі класи) = 0.677
Recall (mean, усі класи) = 0.499
mAP50 = 0.466
mAP50-95 = 0.363
```

Подивимось, на яких класах модель найгірше справляється (найнижчий AP — Average Precision для конкретного класу).

Класи з найнижчим AP@0.5 (найгірше детектуються):

```
laptop      AP50 = 0.000
banana      AP50 = 0.000
mouse       AP50 = 0.000
cell phone  AP50 = 0.000
scissors    AP50 = 0.000
```

Класи з найвищим AP@0.5 (найкраще детектуються):

```
horse       AP50 = 0.995
bear        AP50 = 0.995
zebra       AP50 = 0.995
cat         AP50 = 0.995
skis        AP50 = 0.995
```